

Entwicklung einer interaktiven Active-Learning-Pipeline für die visuelle Analyse und Steuerung von Trainingsprozessen in der Fahrzeugklassifikation

PraxisProjekt
30.04.2026

Inhaltsverzeichnis

#	Thema
1	Motivation & Problemstellung
2	System-Überblick & Live-Demo
3	Active Learning Pipeline
4	Technische Architektur — Streamlit-Constraints, Threading-Modell, Zustandsmaschine
5	ML-Kern — Transfer Learning, Verlustfunktionen, UMAP, Kalibrierung
6	Ergebnisse
7	Fazit

1

Fine-grained Visual Recognition

Klassen sind visuell sehr ähnlich — kleine Details entscheiden

2

Manuelles Labeln

Ist teuer, langsam, und skaliert schlecht

3

Informationsgehalt

Nicht jedes Bild ist gleich informativ für das Modell

4

Kernfrage

Kann ein Modell selbst entscheiden, welche Bilder annotiert werden sollen — und dabei mit einem Bruchteil der Labels eine vergleichbare Genauigkeit erreichen?

Beispiel



BMW 3 Series Sedan 2012



BMW 3 Series Coupé 2012

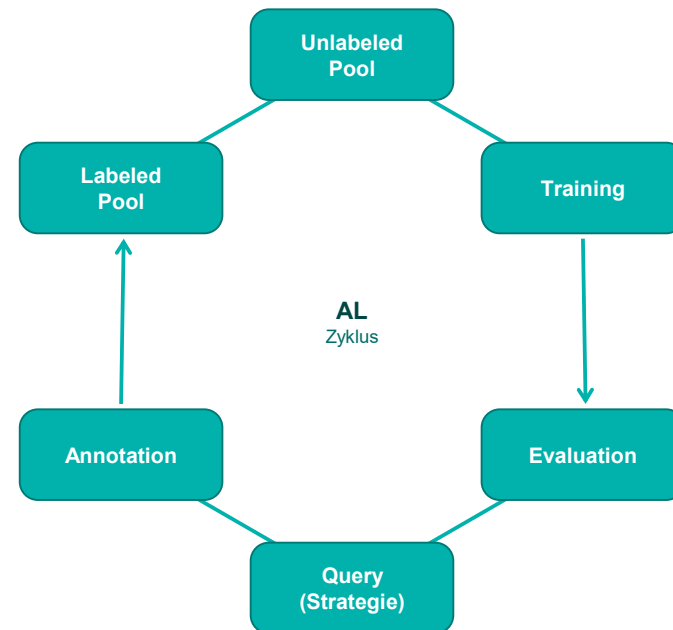
System-Überblick: Interactive Active Learning Framework

⚙️ Konfiguration (Sidebar)

- ▶ Modell: ResNet, MobileNet, EfficientNet
- ▶ Strategie: Entropy, Margin, Least Conf., Random
- ▶ Vollständige Hyperparameter-Kontrolle: Epochs, LR, Scheduler, Freeze-Epochen, Loss-Funktion
- ▶ Step-Mode: Zyklus manuell steuern

▶ Laufzeit (4 Tabs)

- ▶ **Main** — Live Loss/Accuracy, Systemstatus
- ▶ **Annotation** — Queried Bilder anzeigen; simuliertes (Ground-Truth Labels werden automatisch angewendet)
- ▶ **Results** — Accuracy/F1/ECE über Zyklen, UMAP-Embeddings (Plotly), Konfusionsmatrix, Query-Summaries
- ▶ **Compare** — Mehrere Experimente direkt vergleichen: Accuracy, F1, ECE, Summary-Tabelle
- ▶ **Dataset Explorer** — Labeled/Unlabeled Pool durchsuchen



Live Demo

Active Learning Pipeline — Zyklus, Strategien & Implementierung

Das Problem

- 1 Vollständige Annotation ist teuer — nicht jedes Sample gleich wertvoll
- 2 Zufälliges Labeln verschwendet Budget auf redundante Samples

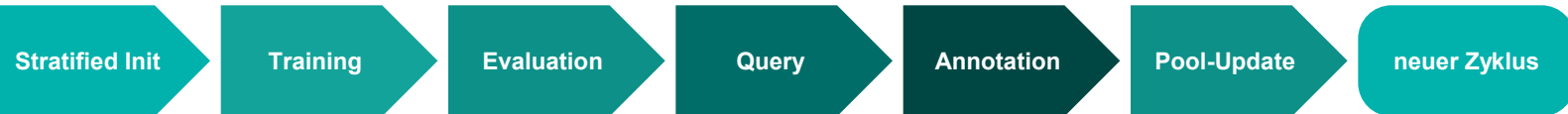
→ Idee: Modell wählt selbst — nur unsicherste Samples annotieren

Technische Besonderheit: Index-basierte Pools

Keine Datenkopien. `_labeled_list` und `_unlabeled_list` sind Integer-Listen.

Pool-Update = nur Indizes verschieben — kein Memory-Overhead, egal wie gross der Datensatz.

Der Zyklus



Schritt	Was passiert im Code	Warum
Stratified Init	<code>ALDataManager: 1/Klasse, L₀=196</code>	Cold-Start; alle Klassen bekannt
Training	<code>train_single_epoch()</code> , Early Stopping	Klassengrenzen auf wachsendem Pool
Evaluation	<code>run_evaluation()</code> → test set, ECE, Temp. Scaling, CycleMetrics speichern	Baseline pro Zyklus
Query	Strategie: bewertet U, wählt Top-k	Gezielte Selektion
Annotation	Oracle (Mensch oder GT-Label)	Einzig teurer Schritt
Pool-Update	<code>update_labeled_pool(): k Indizes U→L</code>	L wächst, U schrumpft

Index-Pools: Kein Memory-Overhead — nur Integer-Indizes werden verschoben, keine Datenkopien. Skaliert beliebig.
`reset_mode='continue':` Gewichte bleiben erhalten — nur Optimizer-State und LR-Schedule werden zurückgesetzt.

Die 4 Query-Strategien:

"Bei 196 Klassen divergieren die drei Uncertainty-Maße: Least Confidence und Margin betrachten nur Top-1/Top-2, Entropy nutzt die gesamte Posterior-Verteilung."

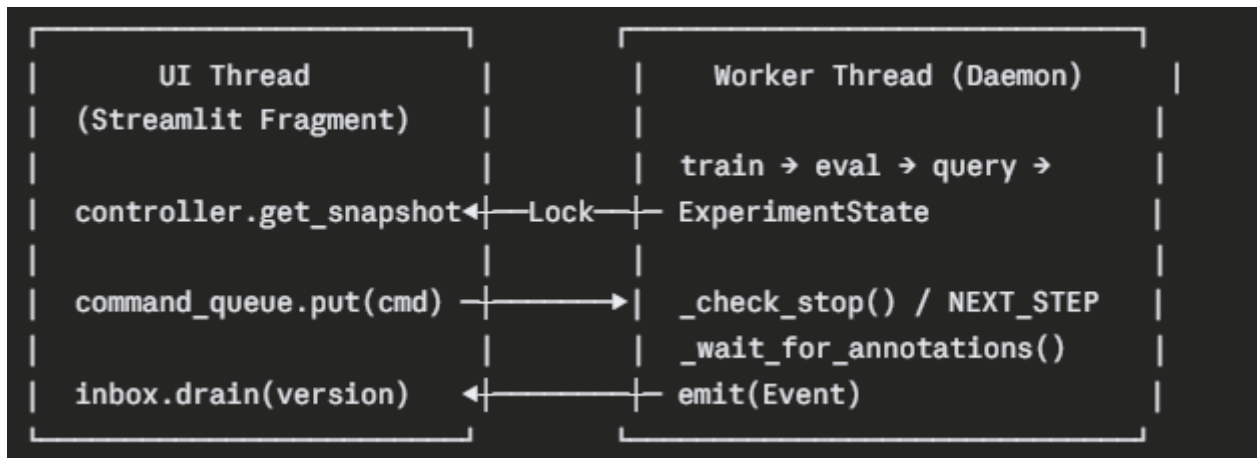
Strategie	Formel	Intuition
Least Confidence	$1 - \max_c P(y=c x)$	Wie unsicher ist das Modell bei seiner Top-1 Klasse?
Margin	$P_{\text{top1}} - P_{\text{top2}}$	Wie nah liegen die zwei wahrscheinlichsten Klassen?
Entropy	$-\sum_c P_c * \log(P_c + \epsilon)$	Wie verteilt ist die Unsicherheit über alle Klassen?
Random	—	Baseline — kein Informationskriterium

Technische Architektur — Streamlit-Constraints, Threading-Modell, Zustandsmaschine

Das Kernproblem: Streamlit ist zustandslos, Streamlit re-rendert das gesamte Script bei jedem User-Klick. Das bricht naiv implementierte ML-Pipelines:

Problem	Lösung im Projekt
Worker-Thread stirbt bei Reload	@st.cache_resource → Controller als Prozess-Singleton
UI blockiert beim Training	Daemon-Thread trennt Training von UI-Thread
Keine Live-Updates ohne Blocking	Adaptives Polling via @st.fragment
Mutable State nicht thread-sicher	Alle Writes unter threading.Lock, UI liest nur Snapshots

Das Zwei-Thread-Modell:



- "Event Inbox (Worker → UI): immutable Event-Dataclass, Version-Counter → Fragment zieht nur neue Events"
- "Command Queue (UI → Worker): plain dicts, non-blocking `get_nowait()` — blockiert nie den UI-Thread"

ML-Kern: Transfer Learning & Fine-Tuning

Modell-Laden via TIMM

- Einheitliche API für ResNet, MobileNet, EfficientNet
- Feature-Dimension: `model.num_features` — automatisch ermittelt
- Embeddings via Forward-Hook auf `model.global_pool` — keine Architektur-Änderung nötig

Freeze → Unfreeze Strategie

Phase / Zyklus

- Zyklus 1, Ep. 1-`freeze_epochs`
- Zyklus 1, danach
- Zyklus 2+ (`continue`)

Was passiert

- Backbone eingefroren — Head trainiert sich sicher ein
- Backbone aufgetaut + Optimizer neu gebaut (Adam-State für alle Params)
- Gewichte behalten — nur Optimizer-State und LR-Schedule reset

Discriminative Learning Rates

Backbone = $0.1 \times \text{LR}$, Head = $1.0 \times \text{LR}$ — pretrained Features stabil halten, Head schnell lernen lassen.

Drei Loss-Modi (konfigurierbar via `loss_fn`):

cross_entropy

CE + label_smoothing
Standard — schnell, stabil

supcon

L_SupCon only
Feature-Qualität maximieren

combined

$(1-\alpha) \cdot CE + \alpha \cdot L_SupCon$
Klassifikation + Struktur

Supervised Contrastive Loss — Idee & Implementierung

Idee: Embeddings gleicher Klasse zusammenziehen, verschiedener Klassen abstoßen — im gesamten Batch, nicht nur paarweise.

- Features L2-normalisiert → Kosinus-Ähnlichkeit
- Samples ohne positives Pair im Batch werden übersprungen
- Besonders relevant bei AL: frühe Zyklen haben wenige Labels pro Klasse — SupCon profitiert von Klassen-Balance im Batch

ProjectionHead — nur für SupCon (kein Query/Eval)

- Ein einziger Forward Pass für CE + SupCon via `global_pool`-Hook — kein doppelter Durchlauf
- ProjectionHead wird **nicht** für Query oder Evaluation genutzt — nur für den Loss

Temperature Scaling — Post-hoc Kalibrierung

Problem: Softmax-Outputs zu confident
→ AL-Uncertainty-Scores verzerrt.

```
# Lernbarer Skalar T, optimiert via  
LBFGS auf Validation Set:  
loss = CrossEntropy(logits /  
T, labels)  
# T > 1 → weichere Verteilung →  
realistischere Unsicherheiten
```

- Kein Retraining — nur ein Skalar wird angepasst
- Messung: ECE (Expected Calibration Error) vor und nach Scaling
- Direkte Wirkung auf AL: kalibrierte Posteriors → Entropy-Query wählt echte Unsicherheiten
- $T > 1$ → Modell war overconfident (Verteilung weicher); $T < 1$ → underconfident (Verteilung schärfer)

UMAP-Embeddings — Visualisierung pro Zyklus

Pipeline nach jeder Evaluation:

```
Backbone → global_pool → [N,D] →  
UMAP (cosine, n_neighbors=15,  
daemon thread) → [N,2]
```

- Labeled Pool: vollständig einbezogen
- Unlabeled Pool: auf 2.000 Samples gecappt (Performance)
- Pool-Encoding: 0=labeled · 1=unlabeled · 2=queried (dieser Zyklus)
- Entropy-Score als 4. Dimension → Farb-Overlay zeigt wo die Strategie fokussiert

`cycle_{n}_embeddings.npz` [Keys: coords, labels, pool, uncertainty] — Läuft im Daemon-Thread — blockiert AL-Zyklus nicht, UI zeigt Ergebnisse sofort

Experimente Setup

Dataset	Stanford Cars — 196 Klassen
Train / Test	~8.000 / ~8.000 (offizieller Split)
Modell	ResNet-50 (ImageNet-pretrained)
Initial Pool	500 (stratified, ≥ 1 /Klasse)
Query-Batch	100 pro Zyklus
Zyklen	30
Epochen / Zyklus	20
Loss	combined: $(1-\alpha) \cdot \text{CE} + \alpha \cdot \text{SupCon}$
Seed	42
Hardware	NVIDIA H100 PCIe, JupyterHub-Cluster

"Vergleich: 4 AL-Strategien (Entropy, Margin, Least Confidence, Random) + voll-überwachte Referenz auf dem gesamten Trainings-Split (200 Epochen, sonst identische Konfiguration)."

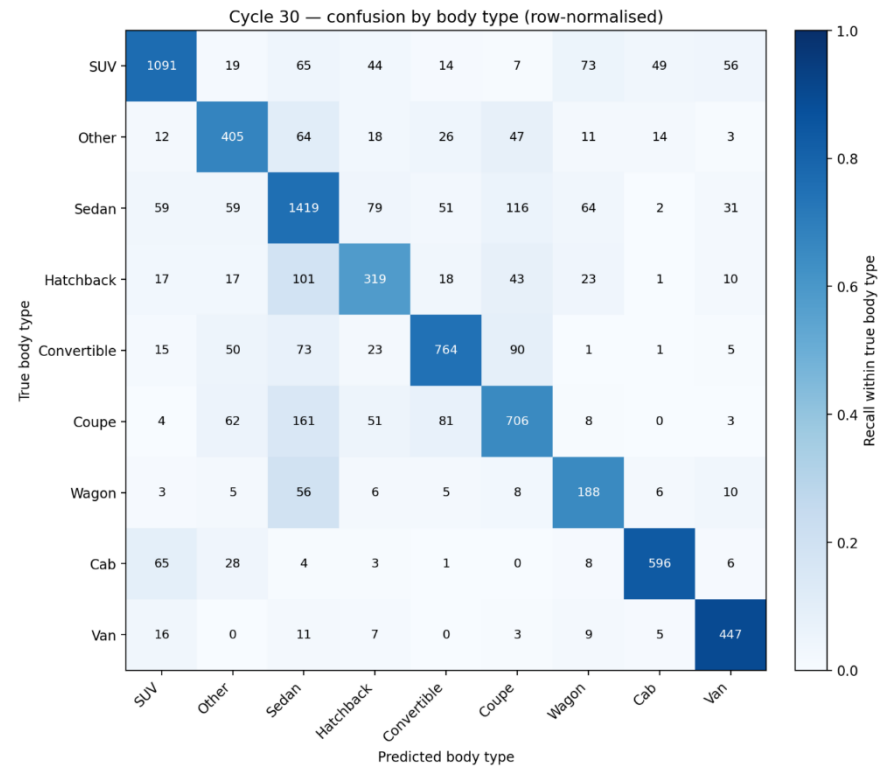
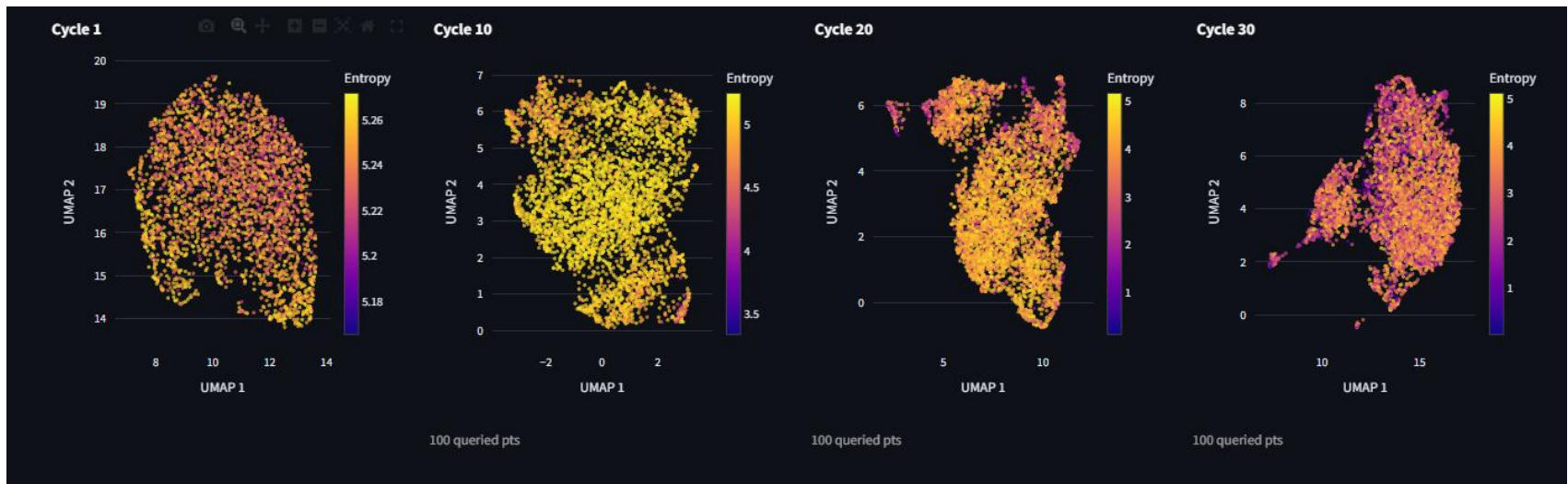
Ergebnisse — 4 Strategien, 30 Zyklen, Stanford Cars



- Entropy > Margin > Least Confidence > Random — Reihenfolge wie bei 196 Klassen erwartet
- AL-Strategien mit 56% der Labels erzielen 68% der voll-überwachten Accuracy
- AL-Läufe besser kalibriert als Supervised-Baseline (ECE 0.16 vs. 0.19) — Temperature Scaling wirkt stärker bei kleinen Pools.

Final Results Summary

Run	Cycles	Final Labeled	Test Acc	F1	ECE
resnet50 / entropy (2026-04-23 14:44)	1	6923	78.06%	0.781	0.1894
resnet50 / least_confidence	30	3900	52.27%	0.516	0.1616
resnet50 / entropy (2026-04-16 02:38)	30	3900	52.93%	0.523	0.1616
resnet50 / margin	30	3900	52.66%	0.522	0.1761
resnet50 / random	30	3900	52.16%	0.517	0.1621



Was wurde gebaut:

- *Voll funktionsfähige interaktive Active-Learning-Pipeline (PyTorch + Streamlit)"*
- *4 Query-Strategien, 3 Loss-Modi, 3 Modell-Architekturen — alle konfigurierbar zur Laufzeit*
- *Ergebnis-Dashboard: Learning Curves, UMAP-Evolution, ECE, Konfusionsmatrix, Run-Vergleich*

Thesis-Beiträge:

- *Empirisch — Uncertainty-Sampling schlägt Random auf Stanford Cars; AL-Läufe besser kalibriert als Supervised-Baseline trotz geringerer Accuracy*
- *Architektonisch — Entkoppeltes Zwei-Thread-Design: SupCon, UMAP, ECE nachträglich integriert ohne eine Zeile UI-Code zu ändern*
- *Didaktisch — Interaktives Tool macht AL-Zyklus, Unsicherheitsmaße und Kalibrierung visuell nachvollziehbar*

FH Aachen
FB5
Amro Elsaadany
3611597

www.fh-aachen.de